Advanced Architectures for Artificial Intelligence in Competitive Pokémon Showdown

The development of artificial intelligence capable of superhuman performance has historically utilized complex games as primary benchmarking environments. While perfect information environments such as Chess and Go have been effectively dominated by algorithmic approaches, the frontier of machine learning has shifted toward environments characterized by imperfect information, high stochasticity, and massive state spaces. Competitive Pokémon Singles (CPS) and the Video Game Championships (VGC) doubles formats, facilitated by the Pokémon Showdown platform, represent extraordinarily complex Partially Observable Markov Decision Processes (POMDP).[1] The computational environment demands sophisticated strategic planning, opponent modeling, resource management, and risk assessment under strict time constraints.[3]

The combinatorial explosion of team configurations is staggering. In the VGC format, each player selects four Pokémon from a roster of six, each configured with customized individual stats, up to four moves, a passive ability, a Tera type, and a held item, resulting in an estimated 10^{139} potential team combinations.[2] When factored across two players, the state space expands to approximately 10^{278}.[2] This immense scale, alongside dynamic battle variables such as stochastic damage rolls, critical hits, hidden abilities, and unrevealed move sets, renders exhaustive search algorithms mathematically intractable and severely limits the efficacy of traditional heuristic models.[2] The transition from hardcoded heuristic algorithms to advanced machine learning paradigms and Large Language Model (LLM) architectures is therefore essential for achieving true human parity or superhuman performance.

State of the Art (SOTA) Review

The landscape of artificial intelligence in Pokémon Showdown has rapidly evolved. Early research predominantly focused on handcrafted rules and shallow search-based methods, such as expectimax or minimax search algorithms augmented with heuristic evaluations.[6] While these approaches established a foundational understanding of the game's mechanics, they consistently failed to adapt to novel metagame shifts or the deceptive attrition strategies employed by expert human players. Recent breakthroughs have transcended these limitations, branching into three highly distinct architectural domains: deep reinforcement learning via self-play, offline sequence modeling, and LLM-augmented game-theoretic search.

Core Architectures and Models

The application of Deep Reinforcement Learning (DRL) has demonstrated significant success in navigating the stochasticity of Pokémon battles. Frameworks utilizing Proximal Policy Optimization (PPO) have been highly prevalent. For instance, specific implementations have deployed a two-headed actor-critic neural network trained through continuous self-play to predict both a policy and a value for a given state.[7] The actor network samples actions while the critic network approximates state values, using these estimates to compute advantages and update the policy. Advanced variants of this approach have augmented the PPO policy with a Monte Carlo Tree Search (MCTS) at test time. The MCTS utilizes the neural network's evaluations to guide its simulated rollouts, compensating for the partial observability of the environment. Such architectures have achieved remarkable rankings, peaking in the top tier of the official Pokémon Showdown generation 4 random battles ladder with an Elo rating exceeding 1690, representing the highest known performance by a non-human agent in that specific format.[1]

Alternatively, the Metamon project introduced an offline reinforcement learning paradigm that eschews traditional Markovian state representations in favor of large sequence models.[9] Utilizing Transformer architectures containing up to 200 million parameters, this approach treats the Pokémon battle as a sequence of discrete tokens, analogous to natural language processing tasks.[10] Because standard Pokémon Showdown replays are recorded from a third-person spectator perspective—obscuring the exact information known to each player at the time of their decision—the Metamon pipeline reconstructs the first-person perspective of an agent from these logs.[9] This allows the sequence model to infer hidden opponent attributes dynamically by attending to the historical context of the battle trajectory. By training on massive datasets of human demonstrations and subsequently fine-tuning via offline reinforcement learning on synthetic self-play data, these models adapt to opponent behavior without requiring explicit heuristic search algorithms, achieving rankings inside the top ten percent of active human players.[9]

The third, and arguably most disruptive, architectural paradigm leverages the zero-shot generalization capabilities of Large Language Models. The PokéLLMon agent achieved human-parity performance by integrating an LLM with specialized prompting frameworks.[12] This architecture consumes text-based feedback derived from the battle engine to continuously refine its generation policy. To combat the hallucination inherent in base LLMs, PokéLLMon utilizes Knowledge-Augmented Generation (KAG) to retrieve external game knowledge, such as type matchups and base power statistics, enabling the agent to make mathematically sound decisions.[12] Furthermore, it employs self-consistency algorithms to mitigate "panic switching"—a phenomenon where the LLM, recognizing a highly disadvantageous state, repeatedly switches Pokémon rather than executing a necessary sacrifice.[12]

Expanding upon the language model paradigm, the PokéChamp framework successfully merged LLMs with a mathematically rigorous minimax tree search. PokéChamp utilizes the

language model for three highly specialized sub-tasks rather than an end-to-end policy.[4] First, it performs player action sampling to aggressively prune the search tree, preventing the exponential branching factor associated with the 107 possible joint actions in double battles.[6] Second, it executes opponent modeling by predicting the most likely countermeasures based on the observed game state. Finally, instead of expanding the search tree to terminal conditions—which is impossible given the game's horizon—it limits expansion to a specific depth and uses the LLM as a value function estimator to score the non-terminal leaf nodes.[4] This approach requires no task-specific weight updates and has demonstrated expert-level performance, achieving an Elo rating of 1500 and consistently outperforming both heuristic engines and frontier models operating without the minimax framework.[4]

State Information and Feature Engineering

The efficacy of any machine learning model in this domain relies heavily on its state representation. The game environment must be vectorized or linguistically encoded to accurately capture the multi-layered complexity of the battlefield.[18] Models operating within the poke-env ecosystem, or utilizing parsed replay formats like Metamon's UniversalState, require dense arrays of highly specific features.[19] Due to the imperfect information constraint, the state representation must dynamically update, utilizing Bayesian inference or recurrent memory to estimate the opponent's hidden attributes as they are gradually revealed during combat.[20]

The following table delineates the exact state information and feature engineering methodologies required for robust model inputs across both vectorized and text-based architectures:

Feature Category	Specific Data Points Required	Vectorized Representation Mechanics	Textual/LLM Representation Mechanics
Active Pokémon Status	Current HP, maximum HP, exact stat values (Attack, Defense, Special Attack, Special Defense, Speed).	Normalized float arrays (0.0 to 1.0) indicating percentage of maximum capacity, alongside integer values for raw stats.	Explicit text strings (e.g., "Charizard is at 45% HP and outspeeds the opponent's Garchomp").
Stat Modifiers	Volatile boosts and drops for all battle stats, including accuracy and	Integer values mapped strictly from -6 to +6, often converted to	Declarative status updates (e.g., "Toxicroak's Attack

	evasion.	multiplier vectors (e.g., $0.25\times$ to $4.0\times$).	is at +2 stages").
Move Properties	Base power, accuracy percentage, elemental type, priority bracket, remaining PP (Power Points), damage category.	Dense categorical one-hot encodings for types and categories; normalized floats for power and accuracy.	JSON-formatted schemas detailing available moves, cross-referenced with external Pokedex databases.
Global Field States	Weather conditions (Sun, Rain, Sandstorm, Snow), Terrains (Electric, Grassy, Misty, Psychic), Trick Room, Tailwind.	Boolean flags indicating presence, paired with integer countdowns representing remaining active turns.	Contextual prefixes appending the active ruleset (e.g., "Heavy rain is falling, reducing Fire-type damage by 50%").
Status Afflictions	Non-volatile conditions (Burn, Paralysis, Poison, Toxic, Sleep, Freeze) and volatile conditions (Confusion, Leech Seed, Taunt).	Multi-class categorical variables or separate boolean masks altering expected utility calculations.	Narrative condition tracking explaining the mechanical impact (e.g., "Gengar is Burned, halving its physical damage output").
Team Composition	Fainted status, revealed bench Pokémon, unrevealed slots, remaining generational mechanics (Dynamax, Terastallization).	Binary masks indicating legal switch targets and boolean flags for available mechanical gimmicks.	Lists of viable replacements, often augmented with Bayesian predictions of unrevealed opponent roster slots.

Vectorized representations typically shape the observation space as a flattened tensor. In formats like VGC doubles, this includes global features, side-specific features, and per-Pokémon features for all twelve entities in the match, frequently utilizing frame-stacking techniques across multiple timesteps to capture temporal dynamics.[21] Conversely, LLM-based agents ingest this exact same data serialized into structured JSON or formatted natural language, relying on the model's self-attention mechanism to weigh the importance of a field condition against a specific stat modifier.[15]

Datasets and Training Data Topologies

The foundation of robust policy learning relies entirely on the acquisition, cleaning, and structural formatting of high-quality training data. Relying on randomly initialized parameters interacting with a simulated environment requires millions of episodes to converge on basic competence. Bootstrapping models with curated professional data significantly accelerates this process.

Existing Public Datasets

The Pokémon Showdown platform generates millions of battle logs on a monthly basis, providing a rich narrative of strategic interactions across various competitive ladders.[22] Several extensive datasets have been parsed from these raw logs to facilitate machine learning research, providing both imitation learning foundations and diverse state-action pairs for reinforcement algorithms.

The Metamon Parsed Replays dataset constitutes one of the most comprehensive resources currently available, encompassing 3.5 million trajectories reconstructed from over a decade of human gameplay.[9] This dataset addresses the fundamental problem of third-person spectator logs by reversing the perspective to reflect exactly what a single player observed at each decision point.[9] Distributed in a highly flexible format via Hugging Face, it utilizes a

UniversalState schema that maps consecutive states (s_t, s_{t+1}) to the corresponding actions executed by the player.[19] This dataset covers formats ranging from Generation 1 to Generation 9 and is specifically tailored for offline reinforcement learning and sequence modeling.[23]

For research focusing on multi-agent interactions and the official tournament ruleset, the VGC-Bench dataset provides over 700,000 battle logs strictly filtered for Open Team Sheets (OTS) in the Double Battle format.[24] Because OTS eliminates a significant portion of hidden information prior to the battle, these logs provide highly accurate state representations without relying heavily on Bayesian inference. The dataset yields over 13 million transitions, processed specifically to support behavior cloning and multi-agent reinforcement learning algorithms like Policy Space Response Oracles (PSRO).[24]

Furthermore, repositories hosted on platforms like HuggingFace, such as the

HolidayOugi/pokemon-showdown-replays dataset, provide access to tens of millions of raw and structured replays, categorized by tier and Elo rating.[27] This dataset is notably available in Parquet format, offering exceptional query performance for massive-scale data ingestion.[29] Smogon's official usage statistics also serve as vital precompiled data, offering probabilistic distributions of teammate pairings, optimal Effort Value distributions, and item frequencies used to inform prior probabilities in Bayesian opponent modeling.[22]

Evaluation of the Parquet Self-Play Pipeline

The proposed strategy involves utilizing existing hardcoded heuristics to execute thousands of automated games against one another, saving the resulting state-action pairs in Parquet format to train a subsequent machine learning model. This methodology is theoretically sound and highly viable, provided specific architectural nuances regarding data diversity and file encoding are rigorously addressed.

The choice of the Apache Parquet format is exceptionally well-suited for processing massive reinforcement learning trajectories. Parquet is an optimized row-columnar storage format designed for high compression ratios and rapid analytical queries.[30] In deep learning pipelines dealing with highly structured, tabular game states—where features include hundreds of sparse boolean flags for field effects and dense one-hot encoded move IDs—Parquet significantly outperforms row-based formats like CSV or JSON.[31] It is frequently more efficient than Protocol Buffers or HDF5 for loading heterogeneous tabular datasets into distributed training environments via Pandas, Polars, or Dask, mitigating bottlenecks in the data loader during GPU-accelerated training.[31]

Generating custom datasets through heuristic self-play effectively resolves the scarcity of perfectly annotated, first-person perspective data. This methodology mirrors the highly successful "SyntheticRLV2" training regimen utilized by the Metamon architecture, which supplemented its baseline human imitation learning dataset with 4 million synthetic replays generated entirely through diverse self-play.[33] By executing agents within the poke-env environment, the exact state dictionaries and the corresponding algorithmic decisions can be perfectly captured without the noise inherent in parsing human replay logs.[34]

However, relying entirely on heuristic self-play introduces a critical risk of policy stagnation and exploitability. Hardcoded heuristics, such as a "Max Damage" player or a simple expectimax algorithm, possess highly deterministic and predictable behavior patterns.[6] An ML model trained exclusively on this synthetic data will rapidly learn to optimize its policy against the specific flaws of those heuristics, but it will fundamentally fail to generalize against the unpredictable, attrition-based, and deceptive strategies deployed by human experts or more advanced generative models.[15]

To counteract this, the self-play environment must strictly enforce high stochasticity. Team

generation must be randomized to cover the widest possible cross-section of the 10^{139} state space, rather than relying on optimal meta teams. Furthermore, the heuristic decision parameters must include epsilon-greedy exploration or temperature scaling, forcing the bots to occasionally take suboptimal actions. Merging this highly diverse, synthetic Parquet dataset with the parsed human trajectories from the Metamon or VGC-Bench datasets will yield a vastly superior, robust training corpus that captures both fundamental mechanical competence and high-level psychological metagames.

Hybrid Architecture Engineering

The transition from purely hardcoded algorithms to machine learning models does not require the wholesale abandonment of established domain knowledge. In environments with hard mechanical rules, deep neural networks often struggle to learn fundamental truths—such as type immunities—requiring millions of episodes to deduce what a human programmer can dictate in a single line of logic. The synthesis of neural network outputs with heuristic logic, constituting a hybrid architecture, offers the highest degree of stability, interpretability, and tactical reliability.[36]

Action Masking Mechanisms

Action masking is a deterministic intervention applied directly to the output of a reinforcement learning policy, designed to completely restrict the agent from exploring logically invalid or strategically disastrous actions.[37] In the context of Pokémon Showdown, the action space fluctuates violently every turn. A Pokémon may be trapped by an opponent's ability, exhausted of Power Points (PP) for a specific move, or holding a Choice item that irrevocably locks it into a single attack for the duration of its presence on the field.

In a standard neural network architecture, the final dense layer produces raw logits representing the mathematical preference for every possible discrete action (e.g., a continuous vector corresponding to all possible moves and bench switches). Action masking intercepts these logits immediately before the softmax activation function is applied. By evaluating the current game state via the poke-env library's battle.available_moves and battle.available_switches properties, the system dynamically generates a binary tensor mask.[35]

Valid actions retain a mask value of 1, allowing their logits to pass through unchanged. Invalid actions are assigned a mask value of 0, and their corresponding logits are mathematically replaced with an exceedingly large negative number (e.g., -1×10^9). When the softmax function subsequently normalizes the distribution, the exponential calculation forces the probability of selecting the masked actions to drop precisely to zero, strictly confining the agent to legal operations.[38]

Beyond merely enforcing the rules of the engine, heuristic action masking is instrumental for

integrating human expertise. Masks can be engineered to strictly prohibit actions known to be suboptimal under specific constraints. For example, the heuristic logic can mask out stat-boosting status moves when the active Pokémon is already at the maximum $+6$ multiplier, or actively mask Electric-type attacks when the opponent is a known Ground-type. This drastically reduces the exploration space, allowing the reinforcement learning algorithm to converge on high-level strategic play exponentially faster, as it no longer wastes computational resources on trial-and-error learning of immutable game mechanics.[37]

Merging Heuristics and Machine Learning via Weighted Ensembles

To effectively combine the subjective "opinions" of a predictive ML model with the rigid calculations of hardcoded heuristics, the architecture must employ dynamic aggregation techniques, frequently formalized as soft voting or weighted ensembling.[40]

In this framework, the machine learning model evaluates the state and outputs a probability distribution across all valid actions. Concurrently, the heuristic engine calculates a deterministic utility score for each action based on damage calculators, speed tier evaluations, and type matchups. To synthesize these distinct data types, the heuristic scores are normalized into a pseudo-probability distribution. The final action selection is determined by calculating a weighted sum of both distributions.

The mathematical formulation allows for dynamic parameter tuning throughout the lifecycle of the agent. During the early stages of deployment or training, the weighting coefficient can heavily favor the heuristic distribution, acting as a "rule shield" that guarantees a baseline level of competence.[36] As the neural network matures, or as its confidence score in a particular state increases, the coefficient dynamically shifts, transferring decision-making authority to the ML model's ability to recognize long-term patterns that the heuristic calculator cannot perceive.

Furthermore, heuristics serve as a critical fallback mechanism within the ensemble. If the machine learning model exhibits critically low confidence—defined by a uniform, high-entropy probability distribution where no single action breaches a predefined statistical threshold—the system can trigger a hard fallback. This transfers absolute control to a robust heuristic, such as a depth-limited expectimax tree search, to resolve the turn.[6] This interplay ensures the AI does not execute irrational maneuvers in highly anomalous, out-of-distribution game states while allowing the ML model to dictate the overarching strategy.

Language Model Agents and Fine-Tuning Pathways

The integration of Large Language Models into turn-based tactical environments capitalizes on their extensive pre-training corpus. Frontier models possess vast amounts of wiki data, competitive forum discussions, and mechanical documentation pertaining to the Pokémon franchise.[16] This latent prior knowledge allows LLMs to function as highly capable zero-shot strategic planning engines, bridging the gap between raw statistical reinforcement and

semantic tactical deduction.

Feasibility of the ReAct Agent Framework

Configuring an LLM as a ReAct (Reasoning and Acting) agent is exceptionally feasible and highly effective for the Pokémon Showdown environment. The ReAct paradigm mandates that the language model generates sequential thought traces prior to selecting a terminal action. This process allows the model to dynamically update its internal strategic plan based on immediate observations, avoiding the rigid, single-step logic that plagues standard prompting.[43] The discrete, turn-based nature of the battle simulator allows the entire game state—including HP percentages, active weather conditions, and stat modifiers—to be losslessly translated into a structured text prompt without the temporal degradation found in real-time continuous control games.[15]

The agentic workflow operates within a closed-loop client-server architecture. The LLM receives the formatted text state and is granted access to a suite of external computational tools. Rather than attempting to hallucinate complex damage calculations or perfectly recall niche type interactions, the ReAct agent generates a thought trace determining that it requires specific mathematical data to proceed. It then invokes a DamageCalculator tool or queries a Pokedex knowledge base via JSON schemas.[46] The deterministic heuristic environment processes the tool call and returns the precise numerical result. The agent incorporates this objective data into its next semantic step before finally executing a FinalizeDecision tool to transmit the chosen move to the Showdown server.

However, deploying ReAct agents faces strict computational limitations that must be engineered around. Official Pokémon Showdown ladders enforce a total clock time of 150 seconds per match, with an incremental time limit of 15 seconds per individual turn.[3] The multi-step generation required by ReAct loops—especially when executing intermediate tool calls and awaiting API responses—can easily exceed this threshold if utilizing massive parameter models hosted on distant cloud infrastructure. Thus, optimizing the prompt context to reduce token count, utilizing parallel tool calling, and deploying smaller, fast-inference models (such as an 8-billion parameter architecture quantized for local execution) is critically required for real-time competitive viability.[4]

Evaluation of the SFT and DPO Fine-Tuning Pipeline

The specific pipeline of taking a small open-source language model, fine-tuning it on the Parquet dataset, utilizing agentic workflows for prompt upgrades, and applying Direct Preference Optimization (DPO) represents the absolute cutting edge of language model alignment for strategic gaming.

The standard methodology for aligning LLMs to complex tasks is Reinforcement Learning from Human Feedback (RLHF). However, RLHF requires training an intermediate reward model to score outputs, followed by optimizing the language model via Proximal Policy Optimization

(PPO). This dual-model process is notoriously unstable, highly sensitive to hyperparameters, and computationally expensive.[48] DPO mathematically bypasses the reward model entirely. By utilizing a parameterization of the Bradley-Terry reward model, DPO allows for the extraction of the optimal policy in a closed form. It optimizes the policy model directly on a dataset of preferences using a simple binary cross-entropy classification loss, actively increasing the log-probability of preferred strategic responses while simultaneously decreasing the likelihood of rejected, suboptimal responses relative to a frozen reference model.[50]

The execution of this pipeline within the Pokémon context is highly structured:

1. **Supervised Fine-Tuning (SFT):** The base LLM is initially trained on the Parquet dataset containing high-Elo human battle logs and heuristic self-play data. This stage is paramount; it teaches the model the fundamental syntax of the game, the structure of the JSON/text state, and the strict output formatting required for move orders. SFT ensures that the subsequent preference data remains in-distribution for the policy model, establishing baseline competence.[50]
2. **Dataset Construction for DPO:** The system ingests the historical trajectories and parses them to extract the exact game state at critical junctions. Agentic workflows—utilizing larger, slower frontier models like GPT-4—can be employed to evaluate these states and construct high-quality preference pairs. For a given prompt representing a battle state, the action executed by the ultimately victorious player, or an action validated by a deep minimax tree search, is labeled as the chosen response. Conversely, the action executed by the losing player, or actions resulting in immediate negative utility (e.g., executing a Water-type move into an opponent with the Water Absorb ability), are labeled as the rejected response.[54]
3. **Direct Preference Optimization Alignment:** The SFT model is then updated using the DPO loss function over the curated pairs. The algorithm maximizes the margin between the chosen and rejected strategic actions.[52]

This specific pipeline effectively addresses the hallucination and strategic inconsistency native to base LLMs. By explicitly providing negative examples through the rejected pairs, the DPO process mathematically penalizes the logits associated with critical blunders. It explicitly teaches the model what *not* to do—such as engaging in excessive "panic switching" or ignoring obvious type immunities—resulting in a highly aligned, strategically coherent agent capable of executing complex attrition strategies and long-horizon planning previously exclusive to top-tier human competitors.[15]

Conclusion and Baseline Architectures

Synthesizing the state-of-the-art research, dataset availability, and advanced algorithmic engineering, two highly distinct baseline architectures emerge as optimal starting points for a production-grade Pokémon Showdown AI. The first prioritizes mathematical rigor and reinforcement learning within a vectorized environment, while the second leverages the

semantic understanding and generalist capabilities of language models.

Baseline 1: Masked Proximal Policy Optimization with Heuristic Ensembles

This architecture focuses on execution speed, absolute mechanical precision, and the exploitation of continuous state spaces. It relies on a deep reinforcement learning agent embedded within a dynamic heuristic framework, making it highly robust for high-frequency ladder climbing and automated self-play environments.

High-Level Implementation Plan:

1. **Environment Instantiation:** Utilize the poke-env library's Gymnasium wrapper to establish the multi-agent reinforcement learning environment. Configure a local, --no-security Pokémon Showdown NodeJS server to interface with the Python client, completely bypassing public server rate limits and facilitating thousands of simultaneous games.[34]
2. **State Vectorization:** Develop a robust observation space function that translates the continuous updates of the Battle object into a normalized, dense 1D tensor. Incorporate multi-class categorical variables for status conditions, one-hot encodings for all 18 elemental types, normalized floats for HP, and specific discrete counters for dynamic field variables like Trick Room or weather.[60]
3. **Action Masking Integration:** Implement a custom forward pass layer within the policy network. Retrieve the battle.available_moves and battle.available_switches properties to construct a dynamic binary mask. Severely penalize the logits of invalid or heuristically irrational actions before the categorical action distribution is sampled, accelerating convergence.[38]
4. **Training Regimen:** Deploy a PPO (Proximal Policy Optimization) algorithm. Initialize training via imitation learning on the Metamon parsed replay dataset, followed by extensive self-play against a rotating pool of deterministic poke-env baselines (e.g., MaxDamagePlayer) to ensure robust exploration and prevent overfitting to a single strategy.
5. **Heuristic Ensemble Layer:** During live inference, implement a soft-voting mechanism. The PPO network outputs a probability distribution over the masked action space. Simultaneously, a heuristic damage calculator evaluates the board and assigns a deterministic utility score to each available move. The final action is executed by taking the argmax of the weighted combination of the PPO probabilities and the normalized heuristic scores.

Baseline 2: DPO-Aligned ReAct Agent with Depth-Limited Minimax

This architecture prioritizes emergent strategic adaptability, zero-shot generalization to novel metagames, and semantic opponent modeling, treating the battle environment as an iterative,

text-based reasoning puzzle.

High-Level Implementation Plan:

1. **Data Engineering:** Parse the Metamon UniversalState human replay dataset alongside custom heuristic self-play logs. Transform these trajectories into an explicit text-based format representing the battlefield narrative. Save these highly structured text pairs into Parquet files using tools like Pandas or Polars to ensure maximum read/write efficiency during the training loop.[19]
2. **SFT and DPO Fine-Tuning:** Select a lightweight, fast-inference open-weight LLM (e.g., an 8B parameter architecture). Perform Supervised Fine-Tuning on the Parquet state-action pairs to instill the fundamental syntax of the Showdown protocol. Subsequently, utilize a frontier model to construct a preference dataset from the logs—delineating game-winning predictions as chosen and mechanical blunders as rejected—and execute Direct Preference Optimization to align the model toward high-level competitive strategies.[53]
3. **Agentic Tooling:** Utilize an orchestrator framework to construct the ReAct loop. Define specialized tools that the LLM can invoke via JSON schemas mid-thought: CalculateDamage(attacker, defender, move), GetSpeedTier(pokemon_a, pokemon_b), and ExecuteAction(action_id).[46]
4. **Minimax Tree Search Integration:** To resolve the LLM's inherent limitations in evaluating long-term horizons, encapsulate the LLM within a depth-limited minimax search tree. Utilize the LLM's probabilistic generation to aggressively sample the top three most likely actions for both the agent and the opponent, effectively achieving dynamic opponent modeling. Expand the game tree by a set depth of turns, and prompt the LLM to evaluate the intrinsic strategic value of the resulting non-terminal leaf nodes.[16]
5. **Execution Loop Optimization:** At the onset of each turn, intercept the raw state data from poke-env, losslessly translate it into the engineered text prompt, execute the Minimax-LLM expansion, allow the ReAct loop to call necessary mathematical tools, and push the final evaluated action back to the Showdown server. This entire asynchronous loop must be strictly optimized via local model quantization and compiled tool logic to execute well within the 15-second incremental timer constraints.[3]

Obras citadas

1. Winning at Pokémon Random Battles Using Reinforcement Learning - DSpace@MIT, fecha de acceso: febrero 20, 2026, https://dspace.mit.edu/handle/1721.1/153888
2. VGC-Bench: Towards Mastering Diverse Team Strategies in Competitive Pokémon - arXiv, fecha de acceso: febrero 20, 2026, https://arxiv.org/html/2506.10326v3
3. PokéChamp: an Expert-level Minimax Language Agent - arXiv, fecha de acceso: febrero 20, 2026, https://arxiv.org/pdf/2503.04094
4. ICML Poster PokéChamp: an Expert-level Minimax Language Agent, fecha de

acceso: febrero 20, 2026, https://icml.cc/virtual/2025/poster/45207
5. VGC-Bench: AI Benchmarking in Pokémon VGC - Emergent Mind, fecha de acceso: febrero 20, 2026, https://www.emergentmind.com/topics/vgc-bench
6. VGC-Bench: A Benchmark for Generalizing Across Diverse Team Strategies in Competitive Pokémon - arXiv, fecha de acceso: febrero 20, 2026, https://arxiv.org/html/2506.10326v2
7. Nash equilibrium estimation in competitive Pokémon using search and supervised learning, fecha de acceso: febrero 20, 2026, https://theses.ubn.ru.nl/server/api/core/bitstreams/b8ddf389-3ed8-474d-b7b5-67f6b97868ad/content
8. Winning at Pokémon Random Battles Using Reinforcement Learning - DSpace@MIT, fecha de acceso: febrero 20, 2026, https://dspace.mit.edu/bitstream/handle/1721.1/153888/wang-jett-meng-eecs-2024-thesis.pdf
9. Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers - RLJ, fecha de acceso: febrero 20, 2026, https://rlj.cs.umass.edu/2025/papers/RLJ_RLC_2025_340.pdf
10. Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers - arXiv, fecha de acceso: febrero 20, 2026, https://arxiv.org/html/2504.04395v1
11. Human-Level Competitive Pokémon via Scalable Offline Reinforcement Learning with Transformers - Paper Detail - Deep Learning Monitor, fecha de acceso: febrero 20, 2026, https://deeplearn.org/arxiv/624557/human-level-competitive-pok%C3%A9mon-via-scalable-offline-reinforcement-learning-with-transformers
12. fecha de acceso: febrero 20, 2026, https://arxiv.org/html/2402.01118v3#:~:text=The%20design%20of%20Pok%C3%A9LLMon%20incorporates,and%20enables%20the%20agent%20to
13. PokéLLMoN, fecha de acceso: febrero 20, 2026, https://poke-llm-on.github.io/
14. New LLM-based AI Agent Achieves Human Performance Levels in Pokémon, fecha de acceso: febrero 20, 2026, https://www.cc.gatech.edu/news/new-llm-based-ai-agent-achieves-human-performance-levels-pokemon
15. PokéLLMon: A Human-Parity Agent for Pokémon Battles with Large Language Models, fecha de acceso: febrero 20, 2026, https://arxiv.org/html/2402.01118v3
16. PokéChamp: an Expert-level Minimax Language Agent - arXiv, fecha de acceso: febrero 20, 2026, https://arxiv.org/html/2503.04094v1
17. PokéChamp: an Expert-level Minimax Language Agent - GitHub, fecha de acceso: febrero 20, 2026, https://raw.githubusercontent.com/mlresearch/v267/main/assets/karten25a/karten25a.pdf
18. Turn-Based Pokémon Battle System - Emergent Mind, fecha de acceso: febrero 20, 2026, https://www.emergentmind.com/topics/turn-based-pokemon-battle-system
19. jakegrigsby/metamon-parsed-replays · Datasets at Hugging Face, fecha de

acceso: febrero 20, 2026, https://huggingface.co/datasets/jakegrigsby/metamon-parsed-replays
20. Official repository of the spotlight ICML 2025 paper, PokeChamp: an Expert-level Minimax Language Agent. - GitHub, fecha de acceso: febrero 20, 2026, https://github.com/sethkarten/pokechamp
21. VGC-Bench: A Benchmark for Generalizing Across Diverse Team Strategies in Competitive Pokémon - arXiv.org, fecha de acceso: febrero 20, 2026, https://arxiv.org/pdf/2506.10326
22. smogon/usage-stats: Scripts for compiling usage stats from the Smogon server - GitHub, fecha de acceso: febrero 20, 2026, https://github.com/smogon/usage-stats
23. README.md · jakegrigsby/metamon-parsed-replays at main - Hugging Face, fecha de acceso: febrero 20, 2026, https://huggingface.co/datasets/jakegrigsby/metamon-parsed-replays/blob/main/README.md
24. cameronangliss/vgc-battle-logs · Datasets at Hugging Face, fecha de acceso: febrero 20, 2026, https://huggingface.co/datasets/cameronangliss/vgc-battle-logs
25. README.md · cameronangliss/vgc-battle-logs at refs/pr/2 - Hugging Face, fecha de acceso: febrero 20, 2026, https://huggingface.co/datasets/cameronangliss/vgc-battle-logs/blob/refs%2Fpr%2F2/README.md
26. cameronangliss/vgc-bench: An AI benchmark for Pokémon VGC with agent implementations using multi-agent reinforcement learning, behavior cloning, LLMs, and heuristics - GitHub, fecha de acceso: febrero 20, 2026, https://github.com/cameronangliss/vgc-bench
27. Datasets - Hugging Face, fecha de acceso: febrero 20, 2026, https://huggingface.co/datasets?other=pokemon
28. HolidayOugi/pokemon-showdown-replays · Datasets at Hugging Face, fecha de acceso: febrero 20, 2026, https://huggingface.co/datasets/HolidayOugi/pokemon-showdown-replays
29. HolidayOugi/pokemon-showdown-replays at refs/convert/parquet - Hugging Face, fecha de acceso: febrero 20, 2026, https://huggingface.co/datasets/HolidayOugi/pokemon-showdown-replays/tree/refs%2Fconvert%2Fparquet
30. Efficient Data Formats for Generative AI: Why Parquet, ORC, and Columnar Storage Matter, fecha de acceso: febrero 20, 2026, https://medium.com/@xiaxiami/efficient-data-formats-for-generative-ai-why-parquet-orc-and-columnar-storage-matter-45f17c47f944
31. Which Data Format to Use For Your Big Data Project? | Towards Data Science, fecha de acceso: febrero 20, 2026, https://towardsdatascience.com/which-data-format-to-use-for-your-big-data-project-837a48d3661d/
32. Best Practice for Data Formats in Deep Learning - SURF User Knowledge Base, fecha de acceso: febrero 20, 2026,

https://servicedesk.surf.nl/wiki/spaces/WIKI/pages/56295498/Best+Practice+for+Data+Formats+in+Deep+Learning
33. jakegrigsby/metamon-synthetic · Datasets at Hugging Face, fecha de acceso: febrero 20, 2026, https://huggingface.co/datasets/jakegrigsby/metamon-synthetic
34. Poke-env: A Python Interface for Training Reinforcement Learning Pokémon Bots — Poke-env documentation, fecha de acceso: febrero 20, 2026, https://poke-env.readthedocs.io/
35. poke-env Quickstart: Practical Examples and Snippets — Poke-env ..., fecha de acceso: febrero 20, 2026, https://poke-env.readthedocs.io/en/stable/examples/quickstart.html
36. Hybrid Rule-Based and Reinforcement Learning for Urban Signal Control in Developing Cities: A Systematic Literature Review and Practice Recommendations for Indonesia - MDPI, fecha de acceso: febrero 20, 2026, https://www.mdpi.com/2076-3417/15/19/10761
37. arXiv:2504.02662v1 [cs.LG] 3 Apr 2025, fecha de acceso: febrero 20, 2026, https://arxiv.org/pdf/2504.02662?
38. Applying Action Masking and Curriculum Learning Techniques to Improve Data Efficiency and Overall Performance in Operational Technology Cyber Security using Reinforcement Learning - arXiv.org, fecha de acceso: febrero 20, 2026, https://arxiv.org/html/2409.10563v1
39. Learning to Play Pokémon Showdown with Reinforcement Learning - CS230, fecha de acceso: febrero 20, 2026, https://cs230.stanford.edu/projects_fall_2018/reports/12447633.pdf
40. Day 8: Ensemble Methods in ML — Voting Algorithm | by Lakshmi M Kanthi | Medium, fecha de acceso: febrero 20, 2026, https://medium.com/@lakshmi.m.kanthi/day-8-ensemble-methods-in-ml-voting-algorithm-9e17e516426b
41. Turn Based Combat - weighted actions? : r/gamedesign - Reddit, fecha de acceso: febrero 20, 2026, https://www.reddit.com/r/gamedesign/comments/1d2bvop/turn_based_combat_weighted_actions/
42. POKÉCHAMP: AN EXPERT-LEVEL MINIMAX LANGUAGE AGENT FOR COMPETITIVE POKÉMON - OpenReview, fecha de acceso: febrero 20, 2026, https://openreview.net/pdf/9ff4aaebc17b3da377d16c204e657cd4947d6fab.pdf
43. A Super Simple ReAct Agent from Scratch | by Sami Maameri | Data Science Collective, fecha de acceso: febrero 20, 2026, https://medium.com/data-science-collective/a-super-simple-react-agent-87913949f69f
44. ReAct - Prompt Engineering Guide, fecha de acceso: febrero 20, 2026, https://www.promptingguide.ai/techniques/react
45. PokéLLMon: A Human-Parity Agent for Pokémon Battles with Large Language Models - arXiv.org, fecha de acceso: febrero 20, 2026, https://arxiv.org/pdf/2402.01118
46. Pokemon Showdown AI Arena - Lakshya Agarwal, fecha de acceso: febrero 20,

2026, https://lakshyaag.com/blogs/building-a-pokemon-arena
47. Using a “Training Agent” to teach Pokémon battle strategy & rare encounter prediction : r/generativeAI - Reddit, fecha de acceso: febrero 20, 2026, https://www.reddit.com/r/generativeAI/comments/1oct3kz/using_a_training_agent_to_teach_pok%C3%A9mon_battle/
48. DPO Trainer - Hugging Face, fecha de acceso: febrero 20, 2026, https://huggingface.co/docs/trl/en/dpo_trainer
49. Direct Preference Optimization: Your Language Model is Secretly a Reward Model, fecha de acceso: febrero 20, 2026, https://arxiv.org/abs/2305.18290
50. Direct Preference Optimization: A Technical Deep Dive - Together AI, fecha de acceso: febrero 20, 2026, https://www.together.ai/blog/direct-preference-optimization
51. A Comprehensive Survey of Direct Preference Optimization: Datasets, Theories, Variants, and Applications - arXiv, fecha de acceso: febrero 20, 2026, https://arxiv.org/html/2410.15595v2
52. Direct Preference Optimization in One Minute - AI Alignment Forum, fecha de acceso: febrero 20, 2026, https://www.alignmentforum.org/posts/7ruzY5LvBqFBWzyMo/direct-preference-optimization-in-one-minute
53. eric-mitchell/direct-preference-optimization - DPO - GitHub, fecha de acceso: febrero 20, 2026, https://github.com/eric-mitchell/direct-preference-optimization
54. Direct Preference Optimization in NeMo RL - NVIDIA Documentation, fecha de acceso: febrero 20, 2026, https://docs.nvidia.com/nemo/rl/0.2.1/guides/dpo.html
55. [D] what's the proper way of doing direct preference optimization (DPO) and why? - Reddit, fecha de acceso: febrero 20, 2026, https://www.reddit.com/r/MachineLearning/comments/1adnq4u/d_whats_the_proper_way_of_doing_direct_preference/
56. Fine-Tuning Techniques - Choosing Between SFT, DPO, and RFT (With a Guide to DPO), fecha de acceso: febrero 20, 2026, https://developers.openai.com/cookbook/examples/fine_tuning_direct_preference_optimization_guide/
57. Direct Preference Optimization (DPO) for LLM Alignment (From Scratch) - GitHub, fecha de acceso: febrero 20, 2026, https://github.com/rasbt/LLMs-from-scratch/blob/main/ch07/04_preference-tuning-with-dpo/dpo-from-scratch.ipynb
58. Direct preference optimization (preview) - Azure OpenAI - Microsoft, fecha de acceso: febrero 20, 2026, https://learn.microsoft.com/en-us/azure/ai-foundry/openai/how-to/fine-tuning-direct-preference-optimization?view=foundry-classic
59. hsahovic/poke-env: A python interface for training Reinforcement Learning bots to battle on pokemon showdown - GitHub, fecha de acceso: febrero 20, 2026, https://github.com/hsahovic/poke-env
60. Reinforcement Learning with Pokemon | Hamish Ivison, fecha de acceso: febrero 20, 2026, https://ivison.id.au/2021/08/02/pokerl.html
61. Reinforcement learning with the Gymnasium wrapper — Poke-env

documentation, fecha de acceso: febrero 20, 2026, https://poke-env.readthedocs.io/en/stable/examples/rl_with_gymnasium_wrapper.html
62. DPO & ORPO — Overview of Preference Alignment algorithms for LLM finetuning. - Medium, fecha de acceso: febrero 20, 2026, https://medium.com/@jakubstrawadev/dpo-orpo-overview-of-preference-alignment-algorithms-for-llm-finetuning-c4837fed0153